

of such failures. Although skip hyperconnections increase the failure-resiliency of distributed DNNs, they do not make the learned weights more prepared for such failures. This is because all neural network components are present during training, as opposed to inference time where some physical nodes may fail. In order to account for the learned weights being more adapted to specific failure scenarios, we introduce failout regularization, which simulates inference-time physical node failure conditions during training.

During training, failout “fails” (i.e., shuts down) a physical node, to make the learned weights more adaptive to such failures and the distributed neural network more failure-resilient. By “failing” a physical node, we mean temporarily removing the neural network components that reside on the physical node, along with all their incoming and outgoing connections. Failout’s training procedure could be done offline, and would not necessarily be employed during runtime. Therefore failing physical nodes would be temporarily removing their neural network components in simulation.

When the neural components of a given physical node shut down using failout, the neural layers of the upstream physical node that are connected to the failing physical node will not receive information from the failing physical node, forcing their weights to take into account this situation and utilize the received information from the skip hyperconnection. In other words, failout forces the information passage through the skip hyperconnections during training, hence adapting the weights of the neural network to account for these failure scenarios during inference.

Formally, consider a neural network which is distributed over V different nodes $v_i, i \in [1, V]$, where for each v_i , we define its failure rate (probability of failure) $f_i \in [0, 1]$. Following this, we define a binary mask b with V components, where its i -th element b_i follows a Bernoulli distribution, with a mean equal to $1 - f_i$, that is $b_i \sim \text{Ber}(1 - f_i)$. During training, for each batch of examples, a new mask b is sampled, and if $b_i = 0$, the neural components of physical node v_i are dropped from computation (v_i ’s output is set to zero in simulated off-line training), thus simulating a real failure scenario. Formally, if Y_i denotes the output of node v_i , then $Y_i = b_i H_i(X_i)$, where $H_i(\cdot)$ is a non-linear transform on X_i , the input of physical node v_i .

Consider a vertically distributed DNN where the nodes are numbered in sequence $1, 2, \dots, V$ from downstream to upstream (cloud). In this setting, for *ResiliNet* we can derive an equation for X_{i+1} , the input to node v_i , as

$$X_{i+1} = Y_i \odot Y_{i-1}, \quad (1)$$

where the operator $\odot$ is defined as: in $X_{i+1} = Y_i \odot Y_{i-1}$, when node v_i is alive $X_{i+1} = Y_i$, and when node v_i fails, $X_{i+1} = Y_{i-1}$. In this definition, we can see that the skip hyperconnection is only active when there is a failure, which corresponds to *ResiliNet*. In the experiments, we also consider a case where skip hyperconnections are always active (we call it *ResiliNet+*), for which eq. (1) is modified to

$$X_{i+1} = Y_i \oplus Y_{i-1}. \quad (2)$$

Although, superficially, failout seems similar to dropout (Srivastava et al. 2014), failout removes a whole segment of

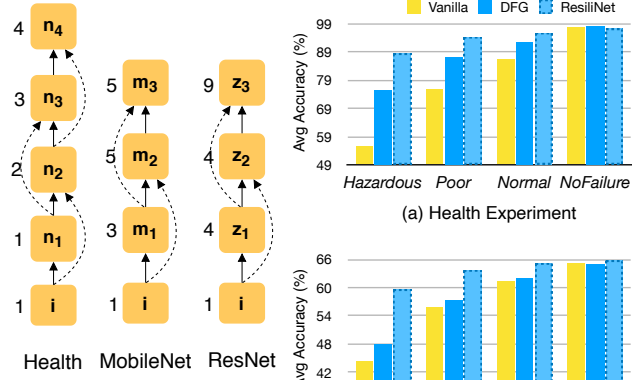


Figure 1: Distributed neural network setup and number of layers on each node.

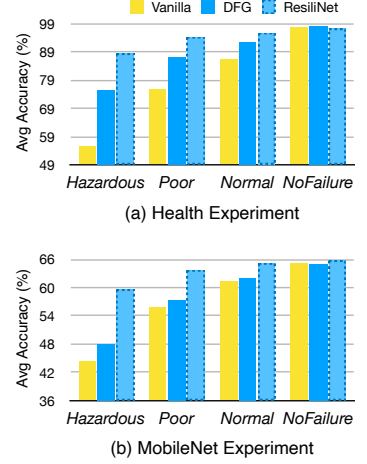


Figure 2: Average performance

neural components, including neurons and weights, for a different purpose of failure-resiliency of distributed DNNs (and not only regularizing the neural network). Another distinction between failout and dropout is in their behavior during inference. In dropout, at inference, the weights of connections are multiplied by the probability of survival of their source neuron to account for model averaging from exponentially many thinned models. Furthermore, DNN units are not dropped during inference in dropout, making the model averaging a necessity. In contrast, failout does not multiply weights of hyperconnections by the probability of survival, since, during inference, physical nodes may fail, though not necessarily at the same rate as during training. Said differently, failout does not use the model ensemble analogy as used in standard dropout, hence does not need the mixed results of the model ensembles. To verify our hypothesis, we conducted experiments using different datasets in a setting where the weights of hyperconnections are multiplied by the probability of survival of the physical nodes, and we observed a sheer reduction in performance.

Experiments

We compare *ResiliNet*’s performance with that of *DFG* (Yousefpour et al. 2019) and *vanilla* (distributed DNN with no skip hyperconnections and no failout).

Scenarios and Datasets

Vertically distributed MLP: This is the simplest scenario for a distributed DNN in which the MLP is split vertically across physical nodes shown in the left of fig. 1.

For this scenario, we use the UCI *health activity classification* dataset (“Health” for short), described in (Banos et al. 2015). This dataset is an example of an IoT application for medical purposes where the inference task will run over a long period of time. The dataset is comprised of readings from various sensors placed at the chest, left ankle, and right arm of 10 patients. There are a total of 23 features, each corresponding to a type of data collected from sensors. For this